

EXPERIMENT NO. 1

Develop a semantic, multi-page technical documentation site

Name: Rohit Pujari

Roll No.: 28

Date: _____

Aim:

To develop the documentation website of “RailTrack”, a practice REST API for Indian Railways timetables, as four linked pages (Overview, Getting Started, Endpoints and Errors & FAQ) written in semantic HTML5 with landmarks, breadcrumbs, a skip link, accessible data tables, definition lists and disclosure widgets, and to check every page automatically with a script that tests the heading outline, the landmarks, the table headers and all internal links.

Theory:

Semantic HTML means choosing each element for what the content is, not for how it should look. A `<div>` or `<span>` says nothing about its content, while `<nav>`, `<main>`, `<table>` or `<dl>` tell the browser that the content is a menu, the main part of the page, tabular data or a list of terms. Browsers build an accessibility tree from these meanings, which screen readers use to jump between regions and headings; search engines and reader modes use the same information. The look is added separately with CSS, so the markup stays clean and easy to maintain.

Page structure and landmarks

Element	Use in the RailTrack docs
<code><header></code>	Site banner with the logo link and the API version written as <code><data value="2.1"></code>
<code><nav aria-label="Documentation"></code>	Side menu of the four pages; the current page has <code>aria-current="page"</code>
<code><nav aria-label="Breadcrumb"></code>	An ordered list (Docs / Endpoints) showing where the page is; the two <code><nav></code> elements are told apart by their <code>aria-label</code>
<code><main id="content"></code>	The one main region of each page; target of the skip link
<code><section aria-labelledby></code>	Each section is named by the id of its own <code><h2></code>
<code><aside></code>	A side note about rate limits that is related to, but not part of, the main text
<code><footer></code> , <code><address></code> , <code><time></code>	Contact of the maintainers and a machine-readable update date

Content elements chosen for documentation

- **`<dl>`, `<dt>`, `<dd>` and `<dfn>`:** name-value pairs such as the key terms and the status codes; `<dfn>` marks the place where a term is defined.
- **`<ol>` versus `<ul>`:** the setup steps use an ordered list because the order matters; the list of possible applications uses an unordered list.
- **`<figure>`, `<pre>`, `<code>`, `<samp>`, `<kbd>`:** code listings are figures with a `<figcaption>`; `<code>` is code to type, `<samp>` is output from the program and `<kbd>` is a key on the keyboard.
- **`<abbr title>`:** gives the full form of REST, JSON and UTC when the pointer rests on the word.
- **`<details>` and `<summary>`:** each FAQ answer is a native disclosure widget that opens with mouse or keyboard without any JavaScript. The link `faq.html#limits` opens the page at the matching answer.

Accessible data tables

The endpoint table has a `<caption>`, column headers in `<thead>` with `scope="col"` and the first cell of each row written as `<th scope="row">`. A screen reader can then announce every cell together with its row and column header, for example “GET, Path, /trains/{number}”.

Headings, skip link and metadata

Every page has exactly one `<h1>`, `<h2>` for its sections and no skipped levels, so the heading outline is a correct table of contents. A “Skip to content” link is the first focusable element; it is moved off-screen with CSS and appears when it receives keyboard focus. Each page has `lang="en"`, its own `<title>` and a meta description for search results.

Consistent pages and automatic checks

The small script `make_pages.py` writes the four pages from one layout function, so the header, menu and footer are identical everywhere and `aria-current` is set on the correct menu item automatically. The script `check.js` opens every page in Chromium through Playwright and tests the rules below.

Check	Rule
Language and metadata	<code>lang="en"</code> and a meta description are present
Headings	Exactly one <code><h1></code> ; no level is skipped (for example <code><h2></code> followed by <code><h4></code>)
Landmarks	<code><header></code> , <code><nav></code> , <code><main></code> and <code><footer></code> all exist; the side menu has <code>aria-current</code>
Sections and tables	Every <code><section></code> has <code>aria-labelledby</code> ; every <code><th></code> has a scope
Links	Every internal link points to an existing page and, with <code>#id</code> , to an existing id

Result of `node check.js`: all four pages passed, with 7 internal links per page and no broken link.

Important Code:

HTML - page skeleton (index.html)

```
<body>
  <a class="skip" href="#content">Skip to content</a>
  <header class="top">
    <a class="brand" href="index.html">RailTrack <span>API Docs</span></a>
    <p class="version">Version <data value="2.1">v2.1</data></p>
  </header>
  <div class="layout">
    <nav class="side" aria-label="Documentation">
      <ul>
        <li><a href="index.html" aria-current="page">Overview</a></li>
        <li><a href="getting-started.html">Getting Started</a></li>
        <li><a href="endpoints.html">Endpoints</a></li>
        <li><a href="faq.html">Errors & FAQ</a></li>
      </ul>
    </nav>
    <main id="content">
```

HTML - accessible table (endpoints.html)

```
<table>
  <caption>Table 1: Endpoints of RailTrack API v2</caption>
  <thead>
    <tr><th scope="col">Method</th><th scope="col">Path</th><th scope="col">Returns</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">GET</th><td><code>/trains/{number}</code></td><td>Timetable of one
train</td></tr>
    ...
  </tbody>
</table>
```

HTML - FAQ entry and CSS for the current page

```
<details id="limits" open>
  <summary>What are the rate limits?</summary>
  <p>100 requests per minute on the free plan and 1000 on the college plan.</p>
</details>

.side a[aria-current="page"] { border-left-color: #0f766e; color: #0f766e; font-weight: 600;
background: #f0fdfa; }
.skip { position: absolute; left: -999px; }
.skip:focus { left: 12px; top: 8px; background: #fff; padding: 6px 10px; border: 2px solid
#0f766e; }
```

Output:

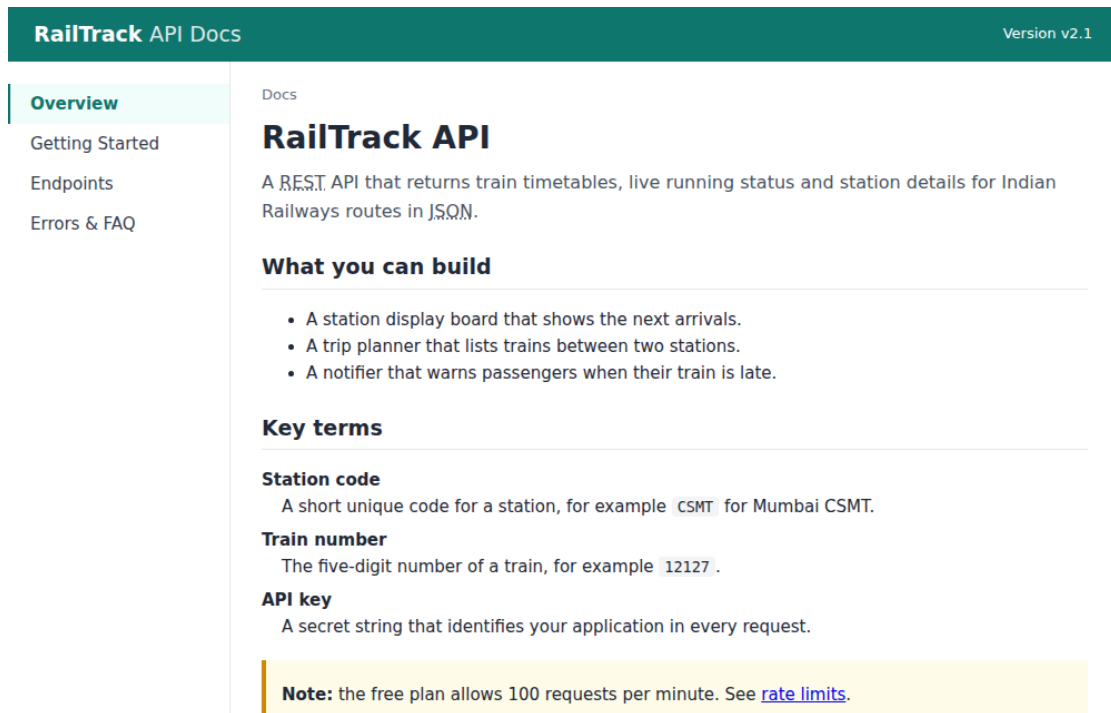


Figure 1.1: Overview page - side menu with aria-current, breadcrumb, <abbr>, <dl> with <dfn> and an <aside> note

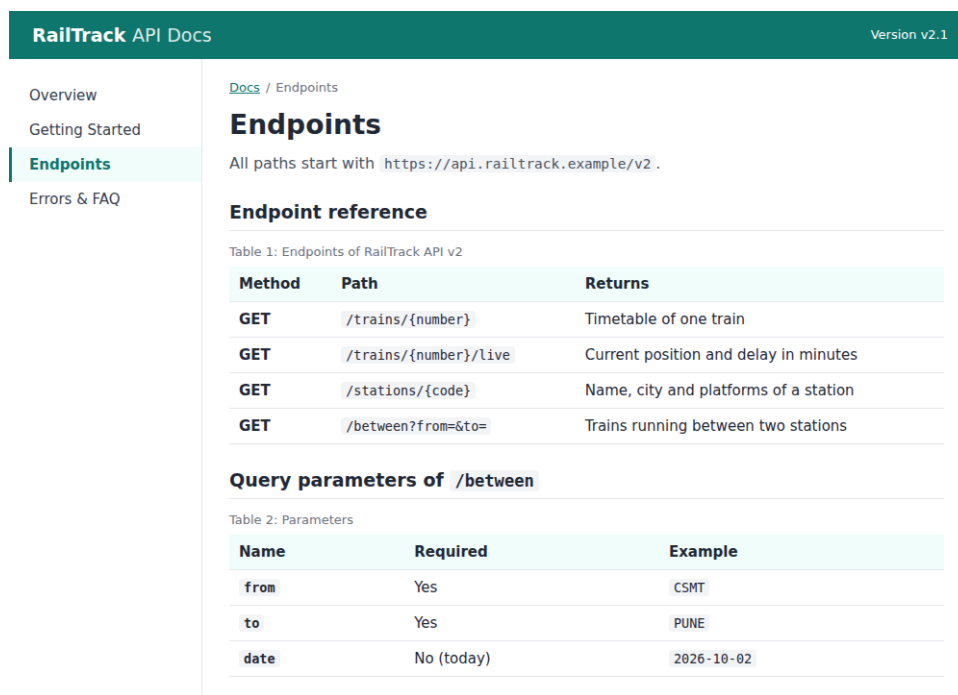


Figure 1.2: Endpoints page - tables with <caption>, column headers and row headers (th scope="row")

Conclusion:

A four-page technical documentation site was developed in semantic HTML5. Landmarks, breadcrumbs, a skip link and a correct heading outline made the pages easy to navigate, while definition lists, figures, accessible tables and <details> widgets gave each kind of content its proper meaning. The automatic check script confirmed that all four pages follow these rules and have no broken links.

EXPERIMENT NO. 4

Implement a dark/light mode toggle using CSS Custom Properties (Variables)

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To build “RecipeBox”, a recipe listing page whose complete colour scheme is stored in CSS custom properties with two sets of values selected by a data-theme attribute on the <html> element; to provide a three-way Light / Dark / System switch that can follow the operating-system setting through matchMedia(), to remember the user's choice in localStorage and to apply the saved theme before the first paint so that the page never flashes in the wrong colours.

Theory:

CSS custom properties (CSS variables) are properties whose names begin with two hyphens, for example --accent. They are declared like any other property, read with the var() function, inherited by child elements and follow the normal cascade, so a more specific rule can give them a new value. Unlike variables of a pre-processor such as Sass, they exist while the page is running, so when their value changes the browser immediately re-styles every element that uses them.

Two variable sets selected by one attribute

RecipeBox declares all its design tokens once on :root. A second rule, :root[data-theme="dark"], has a higher specificity because of the attribute selector and redefines only the colour tokens. No component rule contains a colour code; the header, cards and tags only use var(--surface), var(--text), var(--accent) and so on. Switching the theme therefore means changing a single attribute: document.documentElement.dataset.theme = 'dark'.

Token	Light theme	Dark theme	Used for
--bg	#fbf7f1	#15120f	Page background
--surface	#ffffff	#211c17	Header bar and recipe cards
--text	#2b2118	#f2ebe3	Recipe names and body text
--muted	#7a6a5c	#b0a193	Cooking time and status line
--accent	#b4450c	#fb923c	Logo, selected button, tag text
--border	#e8dccc	#3a3129	Card and header borders
--shadow	soft shadow	none	Card elevation (shadows are hard to see on dark)

Tokens that do not depend on the theme, such as --radius and --gap, are written only once and are shared by both themes. The colour pairs were chosen for readability: body text on the background has a contrast ratio of 14.8:1 in the light theme and 15.8:1 in the dark theme, far above the WCAG AA minimum of 4.5:1.

Three modes: Light, Dark and System

- **Mode and theme are different:** the mode is what the user selected (light, dark or system) and is stored in localStorage; the theme is what is actually applied (light or dark).
- **System mode:** window.matchMedia('(prefers-color-scheme: dark)') returns a MediaQueryList whose .matches property tells whether the operating system is in dark mode.
- **Live OS changes:** the MediaQueryList fires a change event when the user changes the OS setting while the page is open. RecipeBox re-renders only if the saved mode is still “system”, so an explicit Light or Dark choice is never overridden.
- **Accessibility:** the three buttons are inside role="group" with an aria-label, and each button's aria-pressed attribute tells screen readers which mode is selected.

Avoiding the flash of the wrong theme

If the theme were applied by a script at the end of <body>, the browser could paint the light page first and switch a moment later. RecipeBox therefore places a very small inline script inside <head> that reads localStorage and sets data-theme before the body is parsed. The tag <meta name="color-scheme" content="light dark"> tells the browser that both schemes are supported, so scrollbars and form controls are also drawn in the matching style. All localStorage calls are wrapped in try...catch because storage can be blocked, in which case the page falls back to System.

Reading a variable back

`getComputedStyle(document.documentElement).getPropertyValue('--accent')` returns the value that is currently in effect. The status line at the bottom of the page prints the mode, the applied theme and this value, which proves that the same variable name resolves to a different colour in each theme.

Testing the toggle

Test	Expected result	Observed
First visit, OS in light mode	Mode System, light theme applied	As expected
Click "Dark"	data-theme="dark", Dark button pressed	As expected
Reload after choosing Dark	Saved mode read in <head>, page opens dark	As expected
System selected, OS switched to dark	change event switches page to dark	As expected (Fig. 4.2)
Dark selected, OS switched to light	Page stays dark (explicit choice wins)	As expected

Important Code:

CSS - design tokens and the dark variable set

```
:root {
  --bg: #fbf7f1; --surface: #ffffff; --text: #2b2118; --muted: #7a6a5c;
  --accent: #b4450c; --accent-soft: #fde8d7; --border: #e8dccc;
  --shadow: 0 2px 10px rgb(60 30 0 / .08);
  --radius: 12px; --gap: 16px;
}
:root[data-theme="dark"] {
  --bg: #15120f; --surface: #211c17; --text: #f2ebe3; --muted: #b0a193;
  --accent: #fb923c; --accent-soft: #3b2616; --border: #3a3129;
  --shadow: none;
}
```

HTML <head> - apply the saved theme before the first paint

```
<script>
  // Runs before the page is painted, so a saved dark theme never flashes white.
  (function () {
    var mode = 'system';
    try { mode = localStorage.getItem('recipebox-theme') || 'system'; } catch (e) {}
    var dark = mode === 'dark' || (mode === 'system' && matchMedia('(prefers-color-scheme:
dark)').matches);
    document.documentElement.dataset.theme = dark ? 'dark' : 'light';
  })();
</script>
```

JavaScript - switch, save and follow the OS setting

```
function readMode() {
  try { return localStorage.getItem(KEY) || 'system'; } catch (e) { return 'system'; }
}

function render(mode) {
  const theme = mode === 'system' ? (osDark.matches ? 'dark' : 'light') : mode;
  root.dataset.theme = theme; // CSS switches variable set
  buttons.forEach(b => b.setAttribute('aria-pressed', String(b.dataset.mode === mode)));
  const accent = getComputedStyle(root).getPropertyValue('--accent').trim();
  status.textContent = `mode: ${mode} | applied theme: ${theme} | --accent = ${accent}`;
}

buttons.forEach(b => b.addEventListener('click', () => {
  try { localStorage.setItem(KEY, b.dataset.mode); } catch (e) {}
  render(b.dataset.mode);
}));
// When the user follows the OS setting, react to the OS changing while the page is open.
osDark.addEventListener('change', () => { if (readMode() === 'system') render('system'); });
render(readMode());
```

Output:

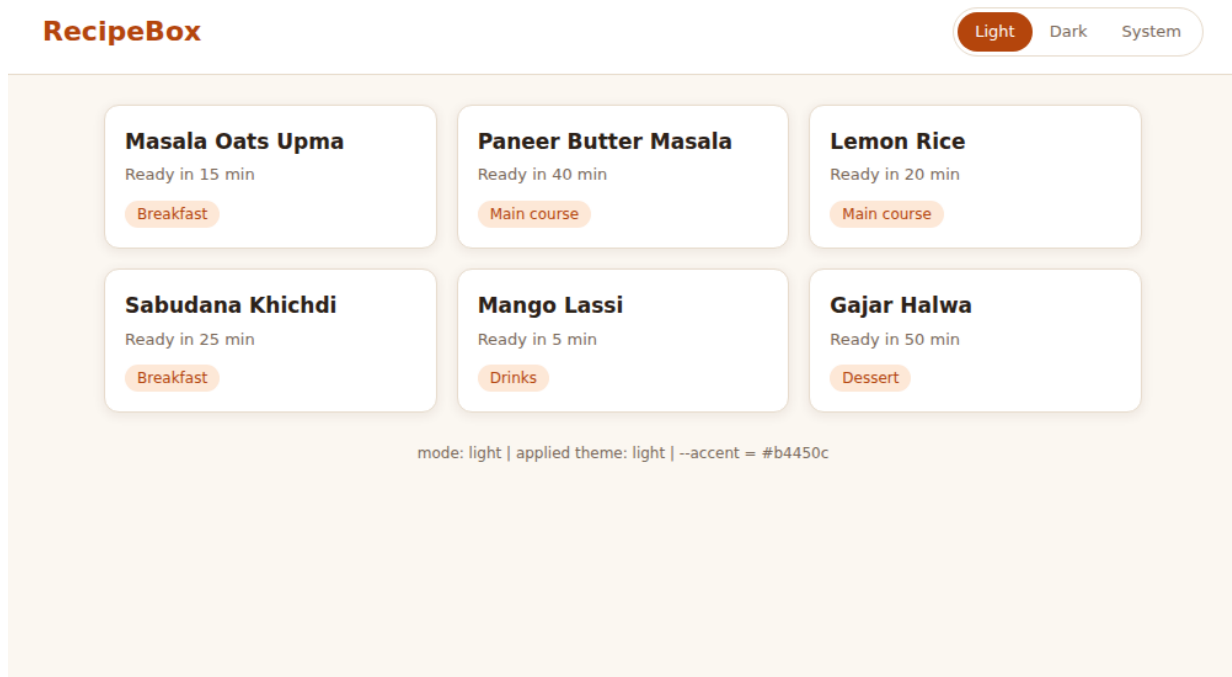


Figure 4.1: RecipeBox with “Light” selected (applied theme light, --accent = #b4450c)

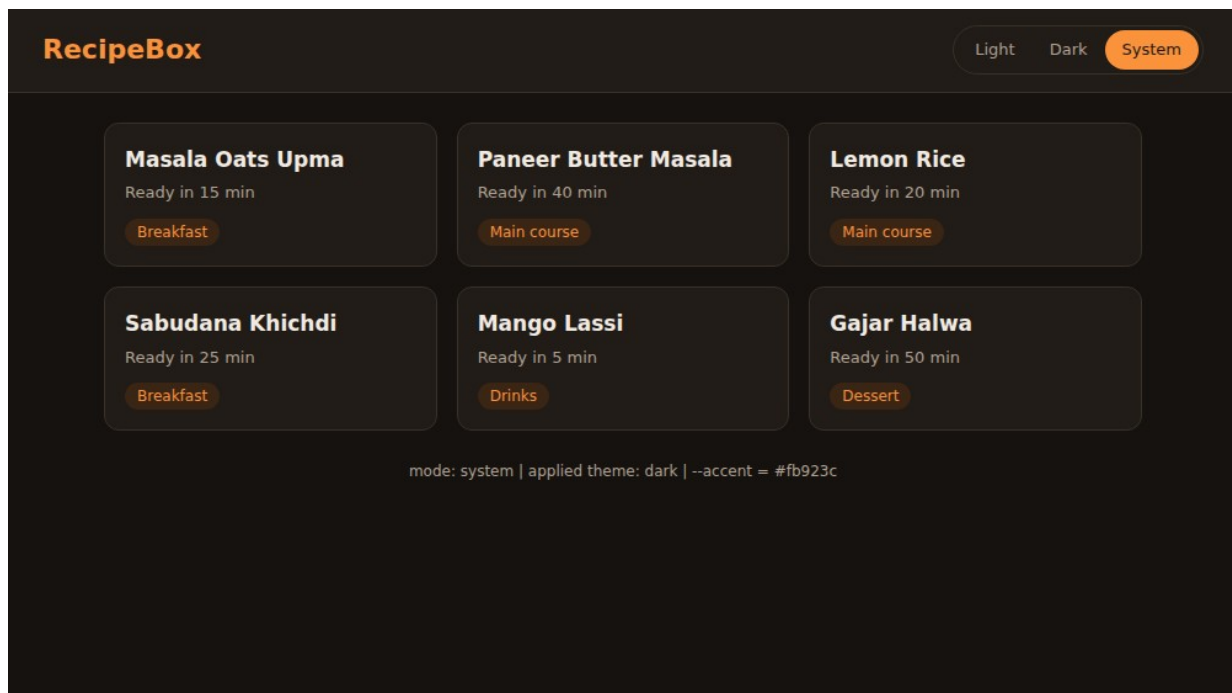


Figure 4.2: “System” selected while the operating system is in dark mode (applied theme dark, --accent = #fb923c)

Conclusion:

A recipe page was themed entirely with CSS custom properties, where a single data-theme attribute on the root element selects the dark set of values. The three-way switch supported Light, Dark and System modes, `matchMedia()` and its change event followed the operating-system preference, `localStorage` remembered the choice and an inline script in `<head>` applied it before the first paint.

EXPERIMENT NO. 5

Build a real-time search filter for a list of items using JavaScript strings/arrays

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To build “BookShelf”, a college library catalogue of 20 books that is filtered on every keystroke by title and author, where a query of several words must match all the words in any order; genre chips generated from the data narrow the list, the result can be sorted by title, year or rating, the matched words are highlighted, and the current search is kept in the page URL so that it can be bookmarked or shared.

Theory:

The catalogue is a JavaScript array of objects, each with title, author, genre, year and rating. The search box listens to the input event, which fires after every character that is typed, pasted or deleted. On each event the program builds a new array with a chain of array methods and redraws the list. The original array is never modified: `slice()` makes a copy before `sort()`, because `sort()` changes the array it is called on.

Multi-word matching with `split()` and `every()`

The query is converted to lower case and split on white space with `split(/\s+/)`; `filter(Boolean)` removes the empty strings produced by extra spaces. For each book the title and author are joined into one lower-case string, and `every(word => text.includes(word))` is true only if all the words occur in it. Therefore “rowling harry” finds both Harry Potter books although the words are in a different order from the title. For an empty query the word list is empty, and `every()` on an empty array returns true, so all books are shown without any special case.

Method	Purpose in BookShelf	Example
<code>split(/\s+/)</code>	Break the query into words	"rowling harry" -> ["rowling", "harry"]
<code>filter(Boolean)</code>	Drop empty strings	["", "code"] -> ["code"]
<code>every(fn)</code>	All words must be present	true for 2 of the 20 books
<code>includes(s)</code>	Substring test	"clean code robert c. martin".includes("code")
<code>[...new Set(arr)]</code>	Unique genres for the chips	20 books -> 7 genres
<code>reduce(fn, 0)</code>	Average rating of the results	$(4.5 + 4.4) / 2 = 4.45$
<code>localeCompare()</code>	Alphabetical sort	"Ikigai" before "Sapiens"
<code>replace(re, "<mark>\$1</mark>")</code>	Highlight matched words	Harry -> <mark>Harry</mark>

Genre chips and sorting

The chips are not written in the HTML; they are generated from the data with `new Set()`, so adding a book of a new genre automatically adds a chip, and the count on each chip is found with `filter().length`. Instead of one listener per chip, a single click listener on the chip container reads `event.target.dataset.genre` (event delegation). The sort order comes from an object of comparator functions: title uses `localeCompare()`, year uses `b.year - a.year` for newest first, and rating sorts by rating and breaks ties by title.

Safe highlighting with regular expressions

The matched words are wrapped in `<mark>` using one regular expression built from all query words, with the flags `g` (all matches) and `i` (ignore case); the capture group `$1` keeps the original capitalisation. Because the words come from the user, characters such as `+` `(` `)` `.` `*` that have a special meaning in a RegExp are first escaped with a backslash, so a query such as “c++ (x)” cannot throw an error. The book text is HTML-escaped before the `<mark>` tags are inserted.

Keeping the search in the URL

- **URLSearchParams** builds a query string such as `?q=code&genre=Programming&sort=rating` from the current state and reads it back when the page loads.
- **history.replaceState()** updates the address bar without reloading the page and without adding one history entry per keystroke, so the Back button still leaves the page.
- **Keyboard shortcuts:** “/” focuses the search box and Esc clears it.

Testing the filter

Test	Expected result	Observed
Type "rowling harry"	2 of 20 books, words highlighted	As expected (Fig. 5.1)
Open ?q=code&genre=Programming	State restored from URL, 1 book (Clean Code)	As expected
Type "xyz"	Message "No books match ..."	As expected
Type "c++ (x"	No error, 0 results	As expected
Press Esc, choose All	All 20 books shown	As expected

Important Code:

JavaScript - escaping and highlighting

```
const escapeRegex = s => s.replace(/[*+?^$(){}|\[\]\\\]/g, '\\$&');

function highlight(text, words) {
  const safe = escapeHtml(text);
  if (!words.length) return safe;
  const re = new RegExp('(' + words.map(w => escapeRegex(escapeHtml(w))).join('|') + ')',
    'gi');
  return safe.replace(re, '<mark>$1</mark>');
}
```

JavaScript - filtering, sorting and the URL

```
function render() {
  const words = state.q.toLowerCase().split(/\s+/).filter(Boolean); // "rowling harry"
  -> 2 words
  const list = books
    .filter(b => state.genre === 'All' || b.genre === state.genre)
    .filter(b => words.every(w => (b.title + ' ' + b.author).toLowerCase().includes(w)))
    .slice().sort(sorters[state.sort]);
  // ... list and chips are rendered here ...
  const p = new URLSearchParams();
  if (state.q) p.set('q', state.q);
  if (state.genre !== 'All') p.set('genre', state.genre);
  if (state.sort !== 'title') p.set('sort', state.sort);
  history.replaceState(null, '', p.toString() ? '?' + p : location.pathname); // shareable URL
}
```

JavaScript - events

```
$('#q').addEventListener('input', e => { state.q = e.target.value; render(); });
$('#sort').addEventListener('change', e => { state.sort = e.target.value; render(); });
$('#chips').addEventListener('click', e => {
  if (e.target.dataset.genre) { state.genre = e.target.dataset.genre; render(); }
});
```

Output:

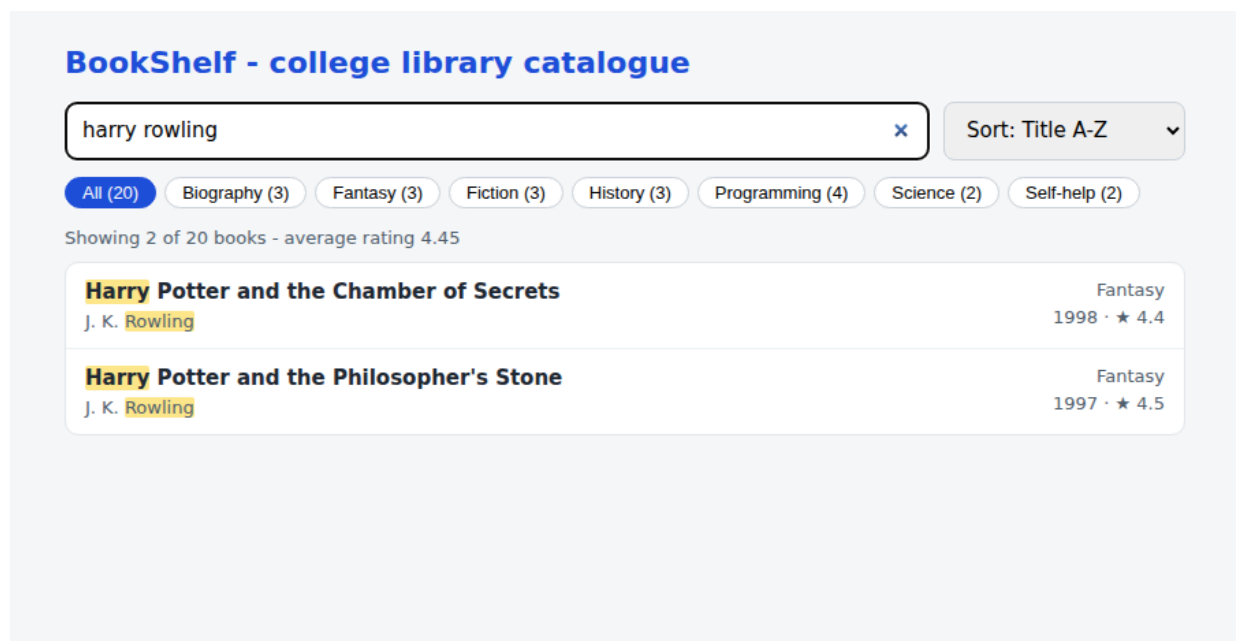


Figure 5.1: Query “harry rowling” - 2 of 20 books match both words, matched words highlighted

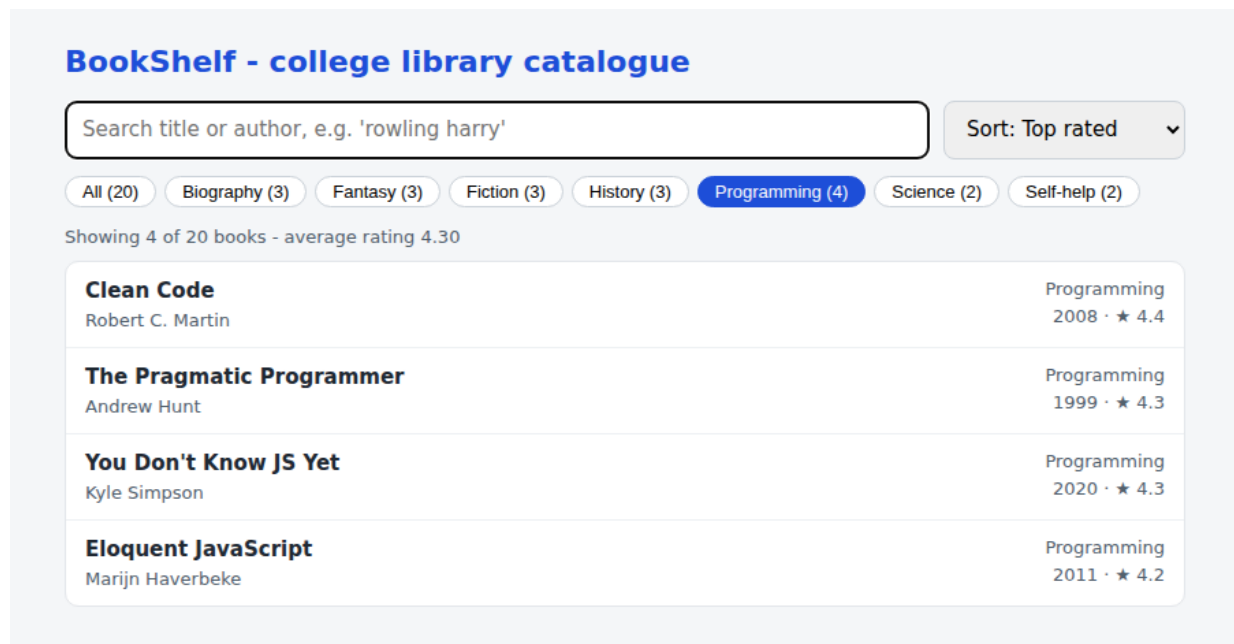


Figure 5.2: Genre chip “Programming” with the sort order “Top rated”

Conclusion:

A real-time book search was built using only JavaScript string and array methods. `split()` and `every()` allowed multi-word queries in any order, `filter()`, `sort()` and `reduce()` produced the result list and its average rating, a Set generated the genre chips, an escaped regular expression highlighted matches safely, and `URLSearchParams` with `history.replaceState()` kept the search in the URL.

EXPERIMENT NO. 8

Implement custom middleware for user authorization in an Express app

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To secure “CampusLibrary”, a library REST API built with Express on port 3008, using only hand-written middleware: a request logger, a signed-token authentication middleware (HMAC-SHA256, without any authentication library), a login-required check, a role-based authorize(...roles) factory, an owner-or-staff check for loan records, a rate limiter on the login route, and central 404 and error handlers.

Theory:

In Express a middleware is a function (req, res, next). It can read or change the request, end the request by sending a response, or call next() to hand control to the next function. Middleware registered with app.use() runs for every request in the order it was added; middleware passed as extra arguments to a route runs only for that route, from left to right, before the final handler. An error handler is recognised by its four parameters (err, req, res, next).

Signed tokens instead of server-side sessions

After a successful login the server returns a token made of two parts: base64url(payload) + "." + base64url(HMAC-SHA256(payload, secret)). The payload contains the user id, the role and an expiry time 15 minutes ahead. The client sends it back in the header Authorization: Bearer <token>. The server recomputes the HMAC with its secret; if even one character of the payload was changed (for example the role “student” replaced by “admin”), the signature no longer matches and the request is rejected. This is the same idea as a JSON Web Token, written by hand.

- **Stateless:** the server keeps nothing per login, so any number of server instances can check a token. The drawback is that a token cannot be cancelled before it expires, which is why the lifetime is short.
- **Encoded, not encrypted:** anyone can read the payload, so it holds no password or secret data.
- **Password storage:** passwords are never stored; crypto.scryptSync() stores a salted hash.
- **Timing-safe comparison:** crypto.timingSafeEqual() compares signatures in constant time, so an attacker cannot guess a signature byte by byte from response times.

The middleware chain

Middleware	Registered	Responsibility	On failure
requestLogger	app.use (global)	Gives each request an id; logs method, URL, status, user and time on the finish event	-
express.json()	app.use (global)	Parses the JSON body into req.body	400 Invalid JSON body
authenticate	app.use (global)	If a Bearer token is present, verifies it and sets req.user; guests pass through	401 Invalid token
requireLogin	per route	req.user must exist	401 Login required
authorize(...roles)	per route	req.user.role must be in the list	403 Forbidden
selfOr(...roles)	/api/loans/:userId	Own user id, or a staff role	403 Forbidden
rateLimit(5, 60000)	/api/login	At most 5 login attempts per IP per minute	429 Too many
404 and error handler	last	Unknown route / unexpected exception	404 / 500

Authentication, authorization and status codes

Authentication answers “who are you?” (token check) and authorization answers “what may you do?” (role and ownership). authorize() and selfOr() are factory functions: calling authorize('librarian', 'admin') returns a new middleware that remembers the allowed roles through a closure. The API uses 401 when the caller is not logged in or the token is invalid, 403 when the caller is known but not allowed, 404 for a missing book or route and 429 for too many login attempts.

Route	Guest	Student	Librarian	Admin
GET /api/books	Yes	Yes	Yes	Yes
GET /api/me	401	Yes	Yes	Yes
GET /api/loans/:userId	401	Own only	Any user	Any user

POST /api/books	401	403	Yes	Yes
DELETE /api/books/:id	401	403	403	Yes

A test script (test.js) calls every route with fetch() as a guest, a student, the librarian and the admin, and also sends a wrong password and a tampered token. Its output is shown in Figure 8.1 and the matching server log in Figure 8.2.

Important Code:

Signing and verifying a token

```
const b64 = s => Buffer.from(s).toString('base64url');
function sign(user) {
  const payload = b64(JSON.stringify({ sub: user.id, role: user.role, exp:
Math.floor(Date.now() / 1000) + TOKEN_TTL }));
  const sig = crypto.createHmac('sha256', SECRET).update(payload).digest('base64url');
  return `${payload}.${sig}`;
}
function verify(token) {
  const [payload, sig] = String(token).split('.');
  if (!payload || !sig) throw new Error('malformed token');
  const expected = crypto.createHmac('sha256', SECRET).update(payload).digest();
  const given = Buffer.from(sig, 'base64url');
  if (given.length !== expected.length || !crypto.timingSafeEqual(given, expected)) throw new
Error('bad signature');
  const data = JSON.parse(Buffer.from(payload, 'base64url').toString());
  if (data.exp < Date.now() / 1000) throw new Error('token expired');
  return data;
}
```

Authentication and authorization middleware

```
function authenticate(req, res, next) { // who are you? (optional)
  const header = req.get('Authorization') || '';
  if (!header.startsWith('Bearer ')) return next(); // guest request
  try {
    const data = verify(header.slice(7));
    req.user = users.find(u => u.id === data.sub);
    if (!req.user) throw new Error('unknown user');
    next();
  } catch (err) {
    res.status(401).json({ error: `Invalid token: ${err.message}` });
  }
}

const requireLogin = (req, res, next) =>
  req.user ? next() : res.status(401).json({ error: 'Login required' });

const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
  : res.status(403).json({ error: `Role '${req.user.role}' cannot ${req.method} $
{req.baseUrl}${req.path}` });

// Owner of the resource, or one of the given staff roles.
const selfOr = (...roles) => (req, res, next) =>
  Number(req.params.userId) === req.user.id || roles.includes(req.user.role) ? next()
  : res.status(403).json({ error: 'You can only view your own loans' });
```

Routes - middleware composed per route

```
app.get('/api/books', (req, res) => res.json(books)); // public
app.get('/api/me', requireLogin, ({ user }, res) => res.json({ id: user.id, username:
user.username, role: user.role }));
app.get('/api/loans/:userId', requireLogin, selfOr('librarian', 'admin'),
  (req, res) => res.json(loans.filter(l => l.userId === Number(req.params.userId))));
app.post('/api/books', requireLogin, authorize('librarian', 'admin'), (req, res) => { ... })
app.delete('/api/books/:id', requireLogin, authorize('admin'), (req, res) => { ... })
```

Output:

```
rohit@laptop: ~/wp-practicals

rohit@laptop:~/wp-practicals$ node exp08/test.js
guest lists books      GET    /books      200    [{"id":101,"title":"Clean Code","copies":3},{ "id":102,"title
guest reads profile    GET    /me         401    {"error":"Login required"}
wrong password         POST   /login      401    {"error":"Wrong username or password"}
login rohit            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOiI..."}
login mehta            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOiI..."}
login admin            POST   /login      200    {"token":"eyJzdWIiOiJlbnVjbGUiOiI..."}
rohit reads profile    GET    /me         200    {"id":1,"username":"rohit","role":"student"}
rohit own loans        GET    /loans/1    200    [{"userId":1,"bookId":101,"due":"2026-10-12"}]
rohit loans of priya   GET    /loans/2    403    {"error":"You can only view your own loans"}
librarian loans of priya GET    /loans/2    200    [{"userId":2,"bookId":102,"due":"2026-10-05"}]
rohit adds book        POST   /books      403    {"error":"Role 'student' cannot POST /api/books"}
librarian adds book    POST   /books      201    {"id":103,"title":"Computer Networks","copies":4}
librarian deletes book DELETE /books/101 403    {"error":"Role 'librarian' cannot DELETE /api/books/101"}
admin deletes book     DELETE /books/101 200    {"id":101,"title":"Clean Code","copies":3}
tampered token         GET    /me         401    {"error":"Invalid token: bad signature"}
```

Figure 8.1: Output of test.js - 200, 201, 401 and 403 responses for guest, student, librarian and admin requests

```
rohit@laptop: ~/wp-practicals

rohit@laptop:~/wp-practicals$ node exp08/server.js
CampusLibrary API on http://localhost:3008
[dcfed88b] GET    /api/books  -> 200  guest      5.5 ms
[0ac4072a] GET    /api/me     -> 401  guest      1.1 ms
[a3f341ce] POST   /api/login  -> 401  guest      62.9 ms
[b5833801] POST   /api/login  -> 200  guest      42.6 ms
[ed6f2da4] POST   /api/login  -> 200  guest      43.4 ms
[a909d2d8] POST   /api/login  -> 200  guest      41.4 ms
[06265b31] GET    /api/me     -> 200  rohit(student) 1.1 ms
[3f628216] GET    /api/loans/1 -> 200  rohit(student) 0.8 ms
[c2d5107f] GET    /api/loans/2 -> 403  rohit(student) 1.1 ms
[bec733c6] GET    /api/loans/2 -> 200  mehta(librarian) 0.6 ms
[fc258b0e] POST   /api/books  -> 403  rohit(student) 0.7 ms
[f9435d35] POST   /api/books  -> 201  mehta(librarian) 0.9 ms
[a80f0926] DELETE /api/books/101 -> 403  mehta(librarian) 0.7 ms
[85628663] DELETE /api/books/101 -> 200  admin(admin) 0.7 ms
[dc8b80e6] GET    /api/me     -> 401  guest      0.5 ms
```

Figure 8.2: Server console written by the requestLogger middleware (request id, status, user and role, time)

Conclusion:

The CampusLibrary API was protected with custom Express middleware only. A hand-written HMAC-signed token identified the user without server-side sessions, authorize() and selfOr() enforced role and ownership rules, and the logger, rate limiter and error handlers completed the chain. The test script confirmed the correct 200, 201, 401 and 403 responses, and a tampered token was rejected.

EXPERIMENT NO. 9

Design a blog database and write scripts to seed it with mock data

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To design the SQLite database of “CodeJournal”, a developer blog with authors, posts, tags and threaded comments (replies to comments); to enforce data rules with constraints, speed up frequent queries with indexes and provide a statistics view; and to write a Node.js seed script (options --posts and --seed, a single transaction with rollback) and a query script that uses joins, aggregation and a recursive CTE, using Node.js's built-in node:sqlite module.

Theory:

Designing a database starts with the entities and their relationships. In CodeJournal an author writes many posts (one-to-many), a post has many tags and a tag belongs to many posts (many-to-many, stored in the junction table post_tags), and a post has many comments (one-to-many). A comment can also answer another comment, which is a one-to-many relationship of the comments table with itself (a self-referencing foreign key).

Tables

Table	Important columns	Constraints and purpose
authors	id, username, full_name, email, joined_on	username and email UNIQUE; CHECK on the e-mail pattern
posts	id, author_id FK, title, slug, body, status, views, published_at	slug UNIQUE (used in the URL); status IN ('draft', 'published'); views >= 0; a published post must have a date
tags	id, name	name UNIQUE COLLATE NOCASE, so "SQL" and "sql" are the same tag
post_tags	post_id FK, tag_id FK	Composite PRIMARY KEY (post_id, tag_id); junction table
comments	id, post_id FK, parent_id FK, name, body, created_at	parent_id NULL for a top-level comment, otherwise the id of the comment being answered

Integrity rules

- **PRAGMA foreign_keys = ON:** SQLite checks foreign keys only when this is switched on.
- **ON DELETE CASCADE:** deleting a post removes its tag links and comments; deleting a comment removes its replies.
- **Table-level CHECK:** CHECK (status = 'draft' OR published_at IS NOT NULL) compares two columns of the same row, which a column constraint cannot do.
- **Normal form:** author names and tag names are stored once and referenced by id, so the design is in third normal form. Dates are ISO text (YYYY-MM-DD), which sorts correctly as text.

Indexes and the view

SQLite automatically indexes primary keys and UNIQUE columns but not foreign keys, so indexes are added on posts(author_id), comments(post_id) and post_tags(tag_id), plus a composite index on posts(status, published_at) for “latest published posts”. EXPLAIN QUERY PLAN confirms that a search by author uses idx_posts_author instead of scanning the table (bottom of Figure 9.2). The view v_post_stats joins posts with authors and uses correlated sub-queries and GROUP_CONCAT() to return the comment count and tag list of every post, so programs can query it like a table.

Seeding strategy

- **Options:** --posts N and --seed S are read from process.argv; the file is deleted and re-created from schema.sql on every run.
- **Reproducible data:** a small seeded random generator (mulberry32) is used instead of Math.random(), so the same --seed always creates exactly the same rows.

- **Realistic rules:** about 80% of posts are published; drafts have no date and 0 views, which satisfies the CHECK constraint. Only published posts get comments, and about 40% of comments get a reply from the author.
- **Prepared statements** are created once and reused for every row, which is faster and safe from SQL injection. INSERT OR IGNORE skips a tag that was already chosen for the same post.
- **One transaction:** all inserts run between BEGIN and COMMIT; on any error ROLLBACK leaves the database empty instead of half-filled. Result: “Seeded codejournal.db (seed=7) in 4.2 ms” (Figure 9.1).

Queries

Query	SQL features	Result
1. Posts and views per author	LEFT JOIN with the condition in ON, GROUP BY, SUM()	Every author listed, busiest first
2. Most used tags	JOIN, GROUP BY, ORDER BY, LIMIT	Top five tags
3. Top posts	SELECT from the view v_post_stats	Title, views, comments, tags
4. Comment thread	WITH RECURSIVE, depth and sort path	Each reply printed under its parent

Important Code:

schema.sql - posts, threaded comments and the view

```
CREATE TABLE posts (
  id          INTEGER PRIMARY KEY,
  author_id   INTEGER NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  slug        TEXT NOT NULL UNIQUE,
  body        TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published')),
  views       INTEGER NOT NULL DEFAULT 0 CHECK (views >= 0),
  published_at TEXT,
  CHECK (status = 'draft' OR published_at IS NOT NULL)      -- published posts need a date
);

CREATE TABLE comments (
  id          INTEGER PRIMARY KEY,
  post_id     INTEGER NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  parent_id   INTEGER REFERENCES comments(id) ON DELETE CASCADE, -- NULL = top-level, else a
  reply
  name        TEXT NOT NULL,
  body        TEXT NOT NULL,
  created_at  TEXT NOT NULL
);

CREATE VIEW v_post_stats AS
SELECT p.id, p.title, a.username AS author, p.status, p.views,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.id) AS comment_count,
       (SELECT GROUP_CONCAT(t.name, ', ') FROM post_tags pt JOIN tags t ON t.id = pt.tag_id
        WHERE pt.post_id = p.id) AS tag_list
FROM posts p JOIN authors a ON a.id = p.author_id;
```

seed.js - seeded random numbers and one transaction

```
let s = Number(args.seed || 7) >>> 0;
const rand = () => { s = (s + 0x6D2B79F5) >>> 0; let t = s; t = Math.imul(t ^ (t >>> 15), t | 1);
  t ^= t + Math.imul(t ^ (t >>> 7), t | 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };

db.exec('BEGIN'); // one transaction for all
rows
try {
  // ... insert authors, tags, posts, post_tags and comments with prepared statements ...
  db.exec('COMMIT');
} catch (err) {
  db.exec('ROLLBACK'); // nothing half-inserted
  console.error('Seeding failed, rolled back:', err.message);
  process.exit(1);
}
```

queries.js - recursive CTE for the comment thread

```
WITH RECURSIVE thread(id, depth, sort_path, name, body) AS (
  SELECT id, 0, printf('%05d', id), name, body FROM comments WHERE post_id = ? AND parent_id
  IS NULL
```

```

UNION ALL
SELECT c.id, t.depth + 1, t.sort_path || '/' || printf('%05d', c.id), c.name, c.body
FROM comments c JOIN thread t ON c.parent_id = t.id)
SELECT substr(' ', 1, depth * 3) || name AS commenter, body FROM thread ORDER BY
sort_path', busiest.post_id);

```

Output:

rohit@laptop: ~/wp-practicals

```

rohit@laptop:~/wp-practicals$ node exp09/seed.js --posts 40 --seed 7 && node exp09/queries.js
Seeded codejournal.db (seed=7) in 4.2 ms

```

(index)	Values
authors	5
tags	8
posts	40
post_tags	75
comments	97

1. Posts and total views per author (published only)

(index)	username	posts	total_views
0	'farah'	9	12890
1	'arjun.dev'	6	7689
2	'vikram'	7	6638
3	'rohitp'	5	5691
4	'sneha_k'	4	3304

Figure 9.1: Seeding with --posts 40 --seed 7 (row counts) and query 1

rohit@laptop: ~/wp-practicals

```

rohit@laptop:~/wp-practicals$ (output of queries.js continued)
2. Most used tags

```

(index)	tag	posts
0	'react'	15
1	'career'	14
2	'css'	11
3	'security'	10
4	'sql'	10

3. Top 5 posts from the view v_post_stats

(index)	title	author	views	comments	tags
0	'Why I Switched to Flexbox'	'farah'	2465	4	'career'
1	'Why I Switched to Database Indexes'	'vikram'	2408	3	'css, sql, react'
2	'5 Mistakes in React Hooks'	'arjun.dev'	2398	4	'security, career'
3	'Deep Dive into Database Indexes'	'farah'	2390	4	'react'
4	'A Beginner Guide to Git Rebase'	'farah'	2085	6	'security'

4. Comment thread of post 30 (recursive CTE)

(index)	commenter	body
0	'Aman'	'Very clear explanation, thanks!'
1	' author'	'Glad it helped!'
2	'Divya'	'Can you add an example with error handling?'
3	' author'	'Good question - I will update the post.'
4	'Divya'	'Very clear explanation, thanks!'
5	' author'	'Node 22, but it works on 20 too.'
6	'Pooja'	'Bookmarked for exams.'

```

Query plan for posts of one author:
SEARCH posts USING INDEX idx_posts_author (author_id=?)

```

Figure 9.2: Queries 2-4 (tags, the view, the recursive comment thread) and the query plan

Conclusion:

A normalised SQLite database for a developer blog was designed with foreign keys, CHECK constraints, a junction table for tags and a self-referencing comments table for replies. Indexes and a view supported the common queries. The seed script generated reproducible mock data inside a single transaction, and the query script demonstrated joins, aggregation, the view and a recursive CTE.

EXPERIMENT NO. 11

Develop a Single Page Application (SPA) dashboard that polls a REST API

Name: Rohit Pujari

Roll No.: 28

Date: _____

Aim:

To develop “ServerPulse”, a single page monitoring dashboard served by Express on port 3011 that polls the REST endpoints `/api/servers` and `/api/servers/:id`, shows the CPU and memory usage of six simulated servers with health badges, opens a detail view with a canvas line chart of CPU history through hash routing, and makes polling robust with `AbortController` time-outs, exponential back-off on errors, a Pause button and automatic pausing when the browser tab is hidden.

Theory:

A Single Page Application loads one HTML page and then updates only parts of it with JavaScript, using JSON data fetched from a REST API. Moving between views does not reload the page. HTTP is a request-response protocol, so the server cannot push new values to the page by itself; the simplest way to keep the dashboard current is polling, i.e. asking the server again at regular intervals. (WebSockets and Server-Sent Events can push data, but need extra server support.)

REST endpoints and simulated data

Method and path	Response	Used by
GET <code>/api/servers</code>	{ time, servers: [{ id, name, region, cpu, mem, status }] }	Overview, every poll
GET <code>/api/servers/:id</code>	One server plus history (last 30 CPU readings); 404 for unknown id	Detail view
GET <code>/</code>	index.html, app.js and style.css via express.static	First page load

On the server, `setInterval()` produces a new reading every 2 seconds. CPU follows a random walk around a base value for each server, and worker-01 is deliberately unreliable and is reported as down about half of the time. The status is calculated from the numbers: down, critical (CPU above 85% or memory above 90%), warning (CPU above 70%) or healthy.

The polling loop

- **Recursive `setTimeout()`:** the next poll is scheduled only after the previous response has been handled, so slow responses never overlap, and the delay can change after every poll. `setInterval()` would fire at a fixed rate regardless.
- **`AbortController`:** each request gets a signal; a 5-second timer aborts a request that hangs, and a new poll (for example after a view change) aborts the one still running.
- **Exponential back-off:** after a failure the delay doubles (3, 6, 12, 24, then at most 30 s) and a red banner shows the retry time; after the next success it returns to 3 s. This avoids flooding a server that is already in trouble.
- **Page Visibility API:** when `document.hidden` becomes true (tab in the background) the `visibilitychange` event stops polling; it restarts immediately when the tab becomes visible.
- **cache: 'no-store'** makes sure the browser never answers a poll from its HTTP cache.

Situation	Next poll
Successful response	after 3 s
Network error, HTTP error or 5 s timeout	previous delay x 2, maximum 30 s
Pause button pressed	none until Resume
Tab hidden	none until the tab is visible again
Hash changed (another view)	immediately

Hash routing and rendering

The views are chosen from `location.hash: "#/server/5".split('/')` gives `["#", "server", "5"]`, so the router knows which endpoint to call. A `hashchange` event starts a new poll at once. The part after `#` is never sent to the server, so no extra server routes are needed, the Back button works and each view can be bookmarked. Cards are built with template literals, `reduce()` counts servers per status for the summary row, and the width of each usage bar is animated with a CSS transition. The detail view draws the CPU history on a `<canvas>`

with the 2D context: grid lines at 0, 50 and 100%, then `moveTo()/lineTo()` for each reading, scaled to the height of the canvas.

Important Code:

app.js - router and polling with back-off

```
function route() {
  const [, page, id] = location.hash.split('/');
  return page === 'server' ? { name: 'detail', id: Number(id) } : { name: 'overview' };
}

async function poll() {
  clearTimeout(timer);
  controller?.abort(); // cancel a request that is still
running
  controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 5000);
  const r = route();
  const url = r.name === 'detail' ? `/api/servers/${r.id}` : '/api/servers';
  try {
    const res = await fetch(url, { signal: controller.signal, cache: 'no-store' });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    r.name === 'detail' ? renderDetail(data) : renderOverview(data.servers);
    lastOk = Date.now(); delay = BASE_DELAY; banner.hidden = true;
  } catch (err) {
    if (err.name === 'AbortError' && !paused) return; // superseded by a newer poll
    delay = Math.min(delay * 2, MAX_DELAY);
    banner.textContent = `Cannot reach server (${err.message}). Retrying in ${delay / 1000}
s...`;
    banner.hidden = false;
  } finally {
    clearTimeout(timeout);
  }
  schedule();
}

function schedule() {
  if (!paused && !document.hidden) timer = setTimeout(poll, delay);
}
```

app.js - pause when the tab is hidden

```
document.addEventListener('visibilitychange', () => { // stop polling in a background tab
  if (document.hidden) clearTimeout(timer); else if (!paused) poll();
});
```

server.js - REST endpoints

```
app.get('/api/servers', (req, res) =>
  res.json({ time: new Date().toISOString(), servers: servers.map(summary) }));
app.get('/api/servers/:id', (req, res) => {
  const s = servers.find(x => x.id === Number(req.params.id));
  if (!s) return res.status(404).json({ error: 'No such server' });
  res.json({ ...summary(s), history: s.history });
});
```

Output:

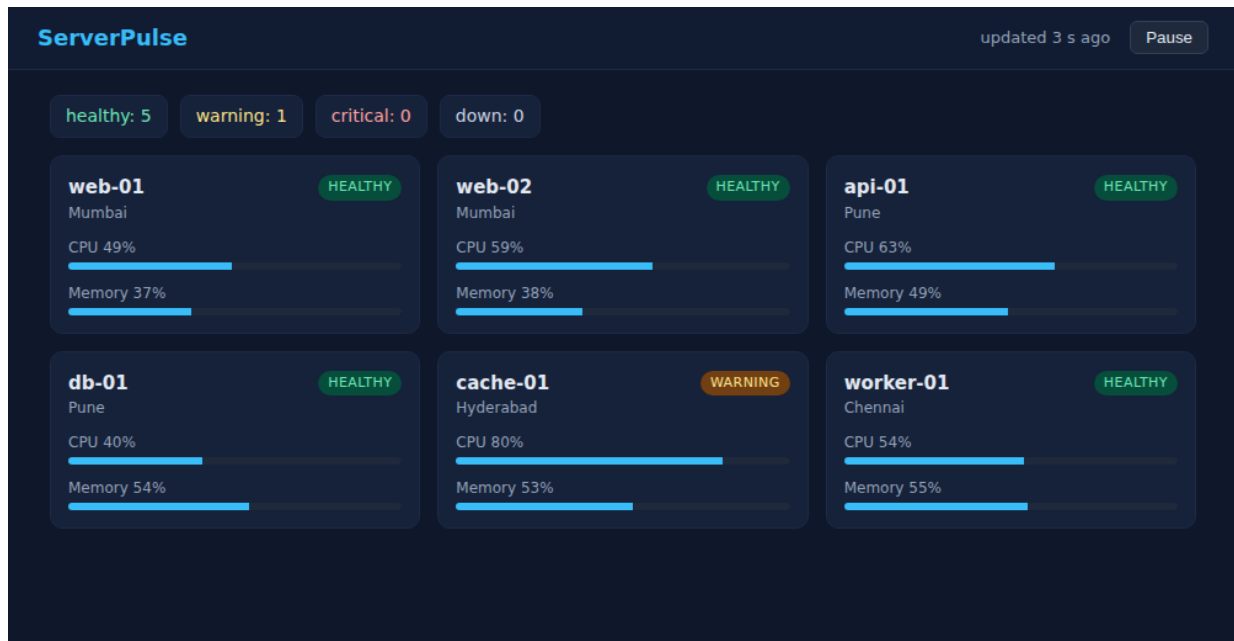


Figure 11.1: Overview view (#/) - live CPU and memory of six servers with the status summary

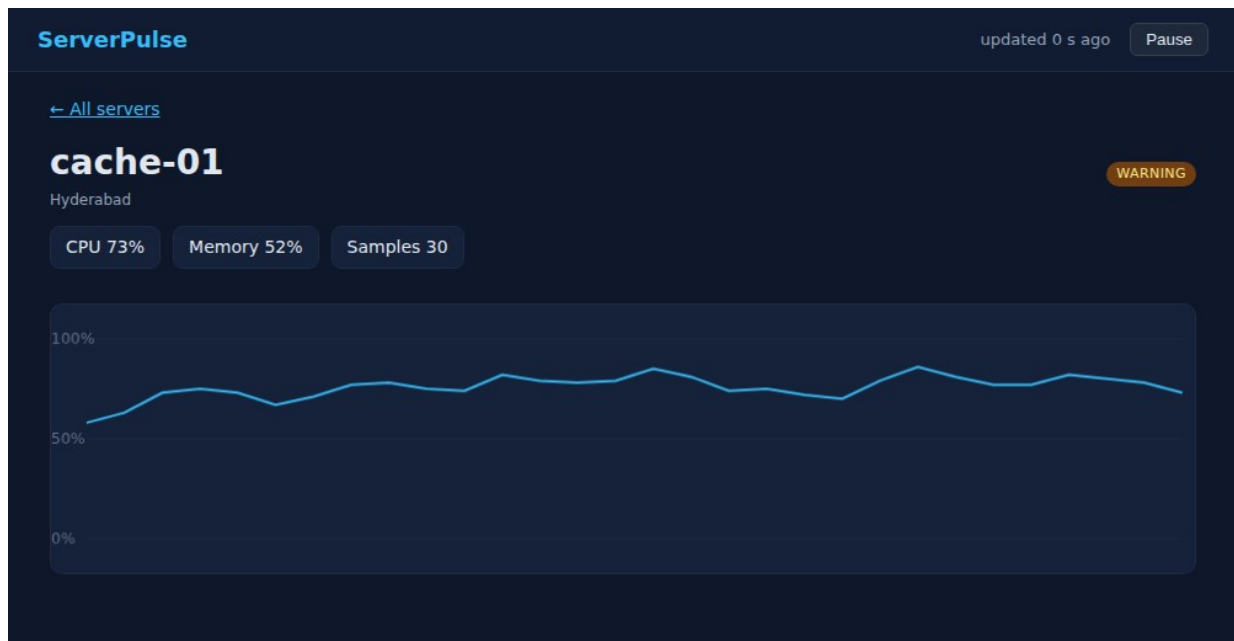


Figure 11.2: Detail view (#/server/5) - CPU history of cache-01 drawn on a canvas

Conclusion:

A single page monitoring dashboard was developed that polls an Express REST API. Hash routing switched between the overview and detail views without page reloads, and the canvas chart showed the CPU history. Recursive `setTimeout()`, `AbortController` time-outs, exponential back-off and the Page Visibility API made the polling efficient and robust.